Especificación de Requisitos de Software (ERS)

Vet-Site - tienda veterinaria

Campo	Detalle
Versión	2.1 - documentación del sistema implementado
Fecha	20 de octubre de 2026
Curso	DSY1104 - Desarrollo Fullstack II
Evaluación	Evaluación Parcial N.º 2
Tipo de sistema	Aplicación frontend demostrativa
Estado del documento	Describe el comportamiento presente en el código del proyecto

1. Introducción

1.1 Propósito

Este documento especifica el alcance, las funciones y las limitaciones de la versión actual de Vet-Site. Sirve como referencia para revisar el código, ejecutar las pruebas frontend y explicar el proyecto durante la presentación de la Evaluación Parcial N.º 2.

Esta versión se documenta según lo implementado en el repositorio. No representa una propuesta de funciones futuras como si ya estuvieran disponibles.

1.2 Alcance del producto

Vet-Site es una tienda veterinaria demostrativa que se ejecuta en el navegador. Permite consultar un catálogo de productos, filtrar por categoría, buscar, operar un carrito local, registrar e iniciar sesión con perfiles de demostración, y gestionar productos, usuarios y solicitudes de compra desde un panel.

Los datos se guardan en `localStorage` del navegador. El sistema no se conecta a una API o base de datos, no sincroniza información entre usuarios o dispositivos y no procesa pagos reales.

1.3 Referencias y precedencia

- DSY1104, *Evaluación Parcial N.º 2: Implementando frontend y pruebas unitarias a mi tienda online* (2024): requisitos de React, Bootstrap, pruebas Jasmine/Karma, cobertura y presentación individual.
- ERS original del Sistema Web Veterinario: requisitos funcionales del caso inicial.
- Código actual del proyecto, ubicado en `src/`, y especificaciones de prueba en `test/unit.spec.jsx`.

Cuando una restricción tecnológica del ERS original difiera de la Evaluación Parcial N.º 2, para esta actualización se sigue la evaluación vigente, que solicita un framework moderno de frontend como React y el uso de Bootstrap.

1.4 Definiciones

- **Cliente:** perfil de demostración que utiliza las funciones públicas de tienda.
- **Vendedor:** perfil de demostración que puede consultar productos y solicitudes desde el panel.
- **Administrador:** perfil de demostración con funciones de mantenimiento de productos y usuarios.
- **Solicitud de demostración:** registro local generado por el checkout; no constituye una compra pagada ni una orden transmitida a un servidor.
- **Persistencia local:** almacenamiento en `localStorage` del navegador actual, asociado al origen desde el que se ejecuta la aplicación.

2. Descripción general

2.1 Perspectiva del producto

La aplicación es un frontend independiente de una sola página (SPA). React actualiza las vistas y el estado; Bootstrap proporciona la grilla y componentes visuales; Vite ejecuta el entorno de desarrollo y genera la compilación.

La navegación se realiza con rutas hash. `App.jsx` coordina la vista actual, la sesión y los estados compartidos. Los componentes reciben datos y funciones mediante props, y usan hooks para el estado local.

2.2 Tipos de usuario

Perfil	Funciones disponibles en esta demostración
Cliente	Ver y buscar productos, filtrar categorías, administrar el carrito, registrarse e iniciar sesión. El acceso al panel muestra acceso restringido.
Vendedor	Consultar la tabla de productos y las solicitudes registradas, incluidos sus detalles. No se muestran controles de mantenimiento de productos ni usuarios.
Administrador	Crear, editar y eliminar productos; consultar órdenes; crear usuarios y modificar o eliminar usuarios.

Los roles y el inicio de sesión se aplican en el frontend. No son controles de seguridad adecuados para proteger información o servicios reales.

2.3 Entorno y dependencias

- Navegador moderno con JavaScript y `localStorage` habilitados.
- React y React DOM.
- Bootstrap 5, importado como dependencia del proyecto.
- Vite para desarrollo y compilación.
- Jasmine y Karma para ejecutar especificaciones frontend en un DOM simulado (jsdom).
- Babel, `karma-webpack` y `babel-plugin-istanbul` para transformar JSX e instrumentar cobertura.

3. Requisitos funcionales implementados

ID	Requisito	Comportamiento implementado	Estado
RF01	Navegación	El menú ofrece Tienda, Nosotros, Consejos, Contacto, acceso y panel. La navegación usa hashes.	Implementado
RF02	Catálogo	Renderiza los productos disponibles en tarjetas con categoría, nombre, descripción, precio, stock e imagen o ícono predeterminado. Permite búsqueda por nombre, descripción o categoría y filtro por categoría.	Implementado
RF03	Carrito	Añade productos, incrementa unidades, limita la cantidad al stock, permite restar o quitar artículos y calcula cantidad y total.	Implementado
RF04	Registro	Solicita RUN, nombre, apellidos, correo, contraseña, fecha de nacimiento opcional, región, comuna y dirección. El selector de comunas depende de la región elegida. El registro público crea clientes; el formulario administrativo permite escoger un rol.	Implementado con regiones precargadas limitadas
RF05	Inicio de sesión	Valida el formato y dominio del correo y la longitud de la contraseña; compara las credenciales con los usuarios guardados en el navegador y deriva según el rol.	Implementado como demostración local
RF06	Contacto	Valida nombre, correo permitido y comentario, y muestra una confirmación local. No transmite el mensaje a un servidor.	Implementado como demostración local
RF07	Mantenimiento de productos	El administrador puede crear, editar y eliminar productos. La validación comprueba código, nombre, categoría, descripción, precio, stock y stock crítico. Los cambios de productos se guardan localmente.	Implementado

ID	Requisito	Comportamiento implementado	Estado
RF08	Permisos por rol	El cliente recibe acceso restringido al panel; el vendedor consulta productos y órdenes; el administrador obtiene controles de mantenimiento.	Implementado en la interfaz frontend
RF09	Usuarios	El administrador puede crear usuarios con rol, cambiar el rol desde la tabla y eliminar usuarios.	Implementado localmente
RF10	Solicitudes de compra	El checkout crea una solicitud local con identificador, fecha, cliente, artículos, cantidades, precios, total y estado de demostración. El panel permite consultar la lista y el detalle.	Implementado como demostración local
RF11	Persistencia	Productos, usuarios, carrito, solicitudes y sesión se guardan y recuperan desde `localStorage`. Si el contenido guardado no se puede interpretar como JSON, la carga usa el valor alternativo definido por la aplicación.	Implementado en el navegador actual

3.1 Reglas de validación

Cuenta e inicio de sesión

- Los dominios permitidos son `@duoc.cl`, `@profesor.duoc.cl` y `@gmail.com`.
- El RUN debe contener entre 7 y 8 dígitos seguidos por el dígito verificador numérico o K; se valida el dígito verificador.
- Nombre y apellidos son obligatorios. Los campos limitan nombre a 50 caracteres y apellidos a 100.
- La contraseña debe tener entre 4 y 10 caracteres.
- Dirección es obligatoria y admite hasta 300 caracteres.
- Las regiones actualmente cargadas son Metropolitana, O’Higgins, Valparaíso y Biobío. Las comunas disponibles dependen de la región seleccionada.
- El correo debe ser único al crear una cuenta.

Contacto

- Nombre: obligatorio, máximo 100 caracteres.
- Correo: obligatorio, formato válido y dominio permitido.
- Comentario: obligatorio, máximo 500 caracteres.
- El mensaje se valida y confirma en el cliente; no se envía externamente.

Productos

- Código: obligatorio, mínimo 3 caracteres y único ignorando diferencias entre mayúsculas y minúsculas.
- Nombre: obligatorio, máximo 100 caracteres.
- Descripción: opcional, máximo 500 caracteres.
- Categoría: obligatoria, seleccionada entre Alimentos, Salud, Higiene, Accesorios y Juguetes.
- Imagen: URL opcional.
- Precio: obligatorio, número igual o mayor que cero; admite decimales.
- Stock: obligatorio, entero igual o mayor que cero.
- Stock crítico: opcional, entero igual o mayor que cero.
- La tabla marca “Stock crítico” cuando el stock actual es menor o igual al valor crítico.

4. Requisitos no funcionales y calidad

4.1 Interfaz y responsividad

La interfaz usa HTML semántico, controles etiquetados, Bootstrap Grid y utilidades responsive, además de reglas CSS propias para pantallas pequeñas. Los mensajes de estado y error usan atributos como `role="status"` o `role="alert"` en los flujos correspondientes.

La responsividad está implementada en el código; este documento no afirma una validación manual en una matriz completa de dispositivos o navegadores.

4.2 Mantenibilidad

La aplicación separa coordinación/estado, componentes visuales, datos iniciales y funciones de carrito, órdenes y validación en archivos distintos. Esto permite modificar responsabilidades sin concentrar toda la interfaz en un solo componente.

4.3 Rendimiento

Las interacciones de la SPA actualizan el estado React y la interfaz sin recargar la página. No se especifica un tiempo máximo medido ni se declara una prueba de rendimiento cuantitativa.

4.4 Seguridad y privacidad

Esta versión es académica y no debe utilizarse con datos sensibles o credenciales reales. Las contraseñas y roles se almacenan en el navegador; el código cliente puede inspeccionarse y modificarse. La validación frontend mejora la experiencia, pero no reemplaza validaciones o autorización en un backend.

4.5 Compatibilidad

El proyecto requiere un navegador moderno compatible con JavaScript, React y `localStorage`. No se declara una certificación de compatibilidad para versiones concretas de Chrome, Firefox, Safari o Edge.

5. Arquitectura y componentes

Archivo o módulo	Responsabilidad
`src/App.jsx`	Rutas hash, estado compartido, usuarios, sesión, carrito, órdenes y sincronización con `localStorage`.
`src/components/SiteHeader.jsx`	Navegación y acceso al carrito.
`src/components/ProductCatalog.jsx`	Búsqueda, filtros y composición de tarjetas del catálogo.
`src/components/ProductCard.jsx`	Presentación individual del producto y acción para añadir al carrito.
`src/components/CartPanel.jsx`	Cantidades, eliminación de artículos, total y checkout de demostración.
`src/components/AuthForms.jsx`	Formularios de acceso, registro y contacto.
`src/components/AdminPanel.jsx`	Productos, usuarios, roles, órdenes y detalles.
`src/data/products.js`	Productos y usuarios iniciales de demostración.
`src/utils/cart.js`	Operaciones puras del carrito y cálculo de resumen.
`src/utils/orders.js`	Construcción de solicitudes locales de demostración.
`src/utils/validation.js`	Validación de cuenta, acceso, productos y formato de precio.
`src/styles.css`	Estilos propios e interacciones responsive complementarias a Bootstrap.
`test/unit.spec.jsx`	Especificaciones Jasmine de utilidades y componentes.
`karma.conf.cjs`	Configuración de Jasmine/Karma, jsdom, Babel e instrumentación de cobertura.

6. Pruebas frontend

El proyecto contiene 29 especificaciones Jasmine ejecutadas por Karma en jsdom. Cubren validaciones de negocio, cálculo del RUN, carrito, solicitudes locales, renderizado de tarjeta, filtros del catálogo, formularios, control de acceso y operaciones administrativas de productos, usuarios y órdenes.

La suite utiliza datos en memoria y un spy de Jasmine para comprobar una interacción del componente. La configuración incluye instrumentación Istanbul y reportes HTML, LCOV y resumen de consola.

`docs/COVERAGE.md` registra una ejecución de 29 pruebas aprobadas y 0 fallidas, con 84,50 % de statements, 79,56 % de branches, 85,52 % de functions y 90,64 % de lines. Estas cifras corresponden a la ejecución local del 20 de octubre de 2026 y deben regenerarse después de cambios relevantes. La suite incluye una prueba del flujo de alta desde el formulario administrativo que verifica la actualización del estado de productos.

7. Limitaciones y funciones fuera de alcance

- No hay backend, API ni base de datos.
- La información local no se comparte entre navegadores o dispositivos y puede borrarse al limpiar los datos del navegador.
- El inicio de sesión no constituye autenticación segura.
- El checkout no procesa pagos ni genera órdenes comerciales reales.
- El formulario de contacto no envía mensajes fuera de la demostración.
- No hay una vista de detalle de producto independiente; la información se presenta en las tarjetas del catálogo.
- La lista de regiones y comunas es una selección precargada, no un catálogo geográfico nacional completo.
- Las funciones de permisos se aplican en la interfaz cliente y no son una barrera de seguridad de servidor.
- La publicación en GitHub y la preparación de un ZIP de entrega son actividades de distribución; este documento no afirma que estén realizadas.

8. Referencias del proyecto

- `README.md`: instalación, ejecución, perfiles de demostración y límites de la aplicación.
- `docs/COVERAGE.md`: matriz de pruebas y resultado de cobertura registrado.
- `docs/PRESENTACION.md`: guion de exposición; debe mantenerse alineado con este ERS y con la versión del código que se presente.